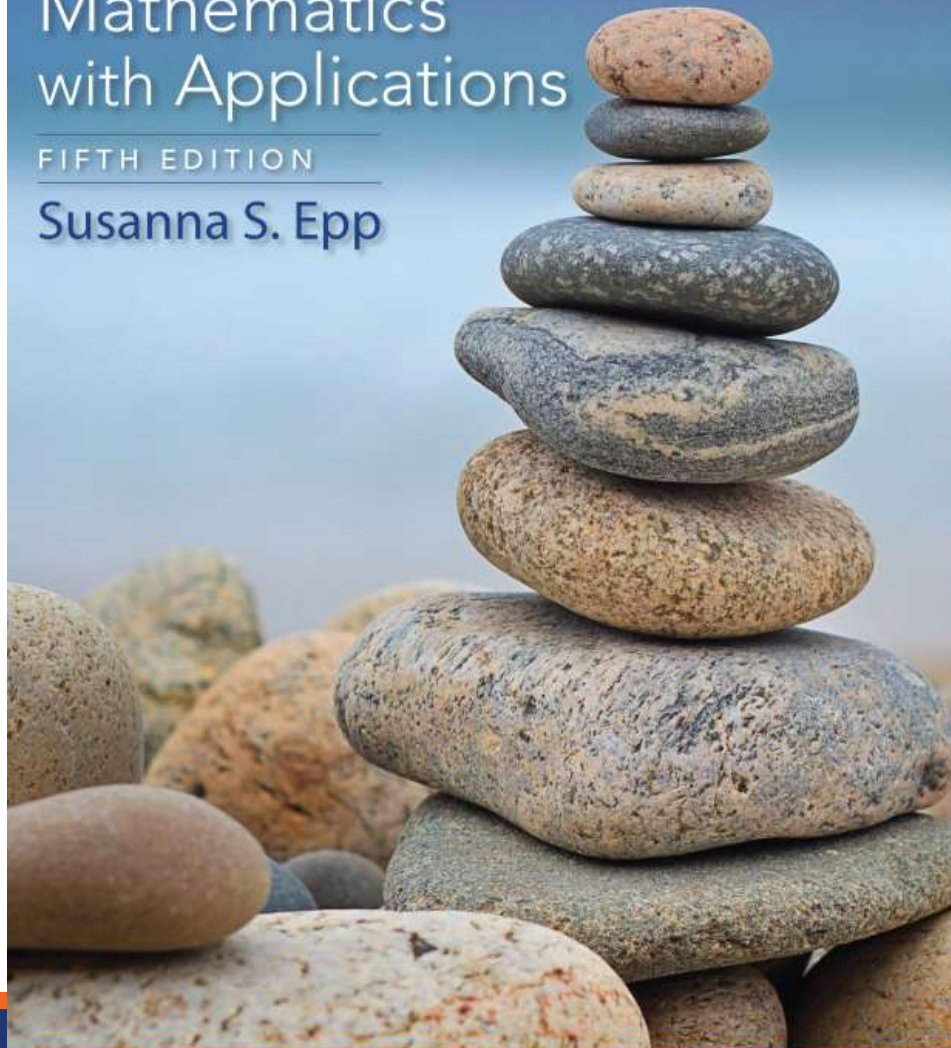


Discrete Mathematics with Applications

FIFTH EDITION

Susanna S. Epp





Faculty of Computer Science

UFCFFS-30-1– Foundations of Computing: REGULAR EXPRESSIONS AND FINITE-STATE AUTOMATA

□ Instructor: Duc Minh Vu

Learning Objectives

1. Formal Languages and Regular Expressions
2. Finite-State Automata
3. Simplifying Finite-State Automata

Reading

- ❑ Chapter 12 - Discrete Mathematics with Applications 5th Edition
(Susanna's Book)
- ❑ Chapter 13 – Discrete Mathematics and Its Applications 7th Edition
(Kenneth Rosen's book)



Formal Languages and Regular Expressions

12.1 Formal Languages and Regular Expressions

- ❑ An English sentence can be regarded as a string of words, and an English word can be regarded as a string of letters
- ❑ Computer languages are similar to English in that certain strings of characters are legitimate words of the language and certain strings of words can be put together according to certain rules to form syntactically correct programs
- ❑ A compiler for a computer language analyzes the stream of characters in a program—first to recognize individual word and sentence units, then to analyze the syntax, or grammar, of the sentences, and finally to translate the sentences into machine code.

Formal Languages and Regular Expressions

□ **Note** A Greek sigma is the traditional symbol used to denote an alphabet in a formal language. It is smaller than the Greek sigma used for a summation

Alphabet Σ:	a finite set of characters
String over Σ:	(1) a finite juxtaposition of elements (called characters) of Σ or (2) the null string λ
Length of a string over Σ:	the number of characters that made up the string, with the null string having length 0
Formal language over Σ:	a set of strings over the alphabet

Example 12.1.1 - Examples of Formal Languages

Examples of Formal Languages

Let the alphabet $\Sigma = \{a, b\}$.

- a. Define a language L_1 over Σ to be the set of all strings that begin with the character a and have length at most three characters. Find L_1 .
- b. A **palindrome** is a string that looks the same if the order of its characters is reversed. For instance, *aba* and *baab* are palindromes. Define a language L_2 over Σ to be the set of all palindromes obtained using the characters of Σ . Write ten elements of L_2 .

Examples of Formal Languages

Let the alphabet $\Sigma = \{a, b\}$.

- Define a language L_1 over Σ to be the set of all strings that begin with the character a and have length at most three characters. Find L_1 .
- A **palindrome** is a string that looks the same if the order of its characters is reversed. For instance, aba and $baab$ are palindromes. Define a language L_2 over Σ to be the set of all palindromes obtained using the characters of Σ . Write ten elements of L_2 .

Solution

- $L_1 = \{a, aa, ab, aaa, aab, aba, abb\}$
- L_2 contains the following ten strings (among infinitely many others):
 $\lambda, a, b, aa, bb, aaa, bab, abba, babaabab, abaabbbbbaaba$



Notation

Let Σ be an alphabet. For each nonnegative integer n , let

Σ^n = the set of all strings over Σ that have length n ,

Σ^+ = the set of all strings over Σ that have length at least 1, and

Σ^* = the set of all strings over Σ .

Note that Σ^n is essentially the Cartesian product of n copies of Σ . The language Σ^* is called the **Kleene closure of Σ** , in honor of Stephen C. Kleene (pronounced CLAY-nee). Σ^+ is the set of all strings over Σ except for λ and is called the **positive closure of Σ** .

Example 12.1.2 The Languages Σ^n , Σ^+ , and Σ^*

Let $\Sigma = \{a, b\}$.

- Find Σ^0 , Σ^1 , Σ^2 , and Σ^3 .
- Let $A = \Sigma^0 \cup \Sigma^1$ and $B = \Sigma^2 \cup \Sigma^3$. Use words to describe A , B , and $A \cup B$.
- Describe a systematic way of writing the elements of Σ^+ . What change needs to be made to obtain the elements of Σ^* ?

Solution

- $\Sigma^0 = \{\lambda\}$, $\Sigma^1 = \{a, b\}$, $\Sigma^2 = \{aa, ab, ba, bb\}$, and $\Sigma^3 = \{aaa, aab, aba, abb, baa, bab, bba, bbb\}$
- A is the set of all strings over Σ of length at most 1.
 B is the set of all strings over Σ of length 2 or 3.
 $A \cup B$ is the set of all strings over Σ of length at most 3.
- Elements of Σ^+ can be written systematically by writing all the strings of length 1, then all the strings of length 2, and so forth.

Σ^+ : $a, b, aa, ab, ba, bb, aaa, aab, aba, abb, baa, bab, bba, bbb, aaaa, \dots$

Of course, the process of writing the strings in Σ^+ would continue forever, because Σ^+ is an infinite set. The only change that needs to be made to obtain Σ^* is to place the null string at the beginning of the list. 

Definition

Let Σ be an alphabet. Given any strings x and y over Σ , the **concatenation of x and y** is the string obtained by writing all the characters of x followed by all the characters of y . For any languages L and L' over Σ , three new languages can be defined as follows:

The **concatenation of L and L'** , denoted LL' , is

$$LL' = \{xy \mid x \in L \text{ and } y \in L'\}.$$

The **union of L and L'** , denoted $L \cup L'$, is

$$L \cup L' = \{x \mid x \in L \text{ or } x \in L'\}.$$

The **Kleene closure of L** , denoted L^* , is

$$L^* = \{x \mid x \text{ is a concatenation of any finite number of strings in } L\}.$$

Note that λ is in L^* because it is regarded as a concatenation of zero strings in L .

New Languages from Old

Let L_1 be the set of all strings consisting of an even number of a 's (namely, λ , aa , $aaaa$, $aaaaaa$, $\dots$), and let $L_2 = \{b, bb, bbb\}$. Find L_1L_2 , $L_1 \cup L_2$, and $(L_1 \cup L_2)^*$. Note that the null string λ is in L_1 because 0 is an even number.

Solution

L_1L_2 = the set of all strings that consist of an even number of a 's followed by b or by bb or by bbb .

$L_1 \cup L_2$ = the set that includes the strings b , bb , bbb and any strings consisting of an even number of a 's.

$(L_1 \cup L_2)^*$ = the set of all strings of a 's and b 's in which every occurrence of a is in a block consisting of an even number of a 's. 



The Language Defined by a Regular Expression

Definition

Given an alphabet Σ , the following are **regular expressions over Σ** :

- I. Base: $\emptyset$, λ , and each individual symbol in Σ are regular expressions over Σ .
- II. Recursion: If r and s are regular expressions over Σ , then the following are also regular expressions over Σ :
 - (i) (rs)
 - (ii) $(r \mid s)$
 - (iii) (r^*) ,
 where rs denotes the concatenation of r and s , r^* denotes the concatenation of r with itself any finite number (including zero) of times, and $r \mid s$ denotes either one of the strings r or s . The regular expression r^* is called the **Kleene closure** of r .
- III. Restriction: Nothing is a regular expression over Σ except for objects defined in (I) and (II) above.

As an example, one regular expression over $\Sigma = \{a, b, c\}$ is

$$a \mid (b \mid c)^* \mid (ab)^*.$$

If the alphabet Σ happens to include symbols—such as $($, or $|$, or $)$, or $*$ —special provisions have to be made to avoid ambiguity. An *escape character*, usually a backslash, is added before the potentially ambiguous symbol. For instance, a left parenthesis would be written as $\backslash($ and the backslash itself would be written as $\\$.

To eliminate parentheses, an **order of precedence** for the operations used to define regular expressions has been introduced. The highest is $*$, concatenation is next, and $|$ is the lowest. It is also customary to eliminate the outer set of parentheses in a regular expression when the order of precedence rules are sufficient for avoiding ambiguity. Thus

$$(a((bc)^*)) = a(bc)^* \quad \text{and} \quad (a \mid (bc)) = a \mid bc.$$

Example 12.1.5

Order of Precedence for the Operations in a Regular Expression

- Add parentheses to emphasize the order of precedence in the following expression:
 $ab^* \mid b^*a$.
- Use the rules about order of precedence to eliminate the parentheses in the following expression: $((a \mid ((b^*)c))(a^*))$.

Solution

- $((a(b^*)) \mid ((b^*)a))$
- $(a \mid b^*c)a^*$

Definition

For any finite alphabet Σ , the function L that associates a language to each regular expression over Σ is defined by (I)–(III) below. For each such regular expression r , $L(r)$ is called the **language defined by r** .

- I. Base: $L(\emptyset) = \emptyset$, $L(\lambda) = \{\lambda\}$, $L(a) = \{a\}$ for every a in Σ .
- II. Recursion: If $L(r)$ and $L(r')$ are the languages defined by the regular expressions r and r' over Σ , then
 - (i) $L(rr') = L(r)L(r')$ (ii) $L(r \mid r') = L(r) \cup L(r')$ (iii) $L(r^*) = (L(r))^*$.
- III. Restriction: The function L is completely determined by I and II above.

Example 12.1.7

The Language Defined by a Regular Expression

Let $\Sigma = \{0, 1\}$. Use words to describe the languages defined by the following regular expressions over Σ .

a. $0^*1^* \mid 1^*0^*$

b. $0(0 \mid 1)^*$

Solution

- The strings in this language consist either of a string of 0's followed by a string of 1's or of a string of 1's followed by a string of 0's. However, in either case the strings could be empty, which means that λ is also in the language.
- The strings in this language have to start with a 0. The 0 may be followed by any finite number (including zero) of 0's and 1's in any order. Thus the language is the set of all strings of 0's and 1's that start with a 0.

Example 12.1.8

Individual Strings in the Language Defined by a Regular Expression

In each of (a) and (b), let $\Sigma = \{a, b\}$ and consider the language L over Σ defined by the given regular expression.

- The regular expression is $a^*b(a \mid b)^*$. Write five strings that belong to L .
- The regular expression is $a^* \mid (ab)^*$. Indicate which of the following strings belong to L :

$a \quad b \quad aaaa \quad abba \quad ababab$

Solution

- The strings b , ab , $abbb$, $abaaa$, and $ababba$ are five strings from the infinitely many in L .
- The following strings are the only ones listed that belong to L : a , $aaaa$, and $ababab$. The string b does not belong to L because it is neither a string of a 's nor a string of possibly repeated ab 's. The string $abba$ does not belong to L because any two b 's that might occur in a string of L are separated by an a . ■

Example 12.1.9

A Regular Expression That Defines a Language

Let $\Sigma = \{0, 1\}$. Find regular expressions over Σ that define the following languages:

- The language consisting of all strings of 0's and 1's that have even length and in which the 0's and 1's alternate.
- The language consisting of all strings of 0's and 1's with an even number of 1's. Such strings are said to have *even parity*.
- The language consisting of all strings of 0's and 1's that do not contain two consecutive 1's.

Solution

- a. If a string in the language starts with a 1, the pattern 10 must continue for the length of the string. If it starts with 0, the pattern 01 must continue for the length of the string. Also, the null string satisfies the condition by default. Thus an answer is

$$(10)^* \mid (01)^*.$$

- b. Some basic strings with even parity are λ , 0, and 10^*1 . Concatenation of strings with even parity also have even parity. Because such a string may start or end with a string of 0's, one answer is

$$(0 \mid 10^*1)^*.$$

- c. Note that a string may end in a 1, but any other 1 must be followed immediately by a 0. Thus, it is enough to enforce the rule that a 1 must be followed by a 0, unless the 1 is at the end of the string. A regular expression satisfying these conditions is

$$(0 \mid 10)^*(\lambda \mid 1).$$

Note that a given language may be defined by more than one regular expression. For example, both

$$(a^* \mid b^*)^* \quad \text{and} \quad (a \mid b)^*$$

define the language consisting of the set of all strings of a 's and b 's.

Example 12.1.10

Deciding Whether Regular Expressions Define the Same Language

In (a) and (b), determine whether the given regular expressions define the same language. If they do, describe the language. If they do not, give an example of a string that is in one of the languages but not the other.

a. $(a \mid \lambda)^*$ and a^*

b. $0^* \mid 1^*$ and $(01)^*$

Solution

- Note that because the null string λ has no characters, when it is concatenated with any other string x , the result is just x : for every string x , $x\lambda = \lambda x = x$. Now $L((a \mid \lambda)^*)$ is the set of strings formed using a and λ in any order, and so, because $a\lambda = \lambda a = a$, this is the same as the set of strings consisting of zero or more a 's. Thus $L((a \mid \lambda)^*) = L(a^*)$.
- The two languages defined by the given regular expressions are not the same: 0101 is in the second language but not the first. 



Practical Uses of Regular Expressions (option)

$[A - C]$ stands for $(A | B | C)$

and

$[0 - 9]$ stands for $(0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9)$.

Character classes are also allowed to include more than one range of characters. For instance,

$[A - C x - z]$ stands for $(A | B | C | x | y | z)$

As an example, consider the language defined by the regular expression

$$[A - Z a - z]([A - Z a - z] \mid [0 - 9])^*.$$

The following are some strings in the language:

Account Number, z23, jsmith109, Draft2rev.

Other commonly used shorthands are

$[ABC]$ to stand for $(A \mid B \mid C)$

and a single dot

$.$ to stand for an arbitrary character.

Thus, for instance, if $\Sigma = \{A, B, C\}$, then

$A.C$ stands for $(AAC \mid ABC \mid ACC)$.

When the symbol $\wedge$ is placed at the beginning of a character class, it indicates that a character of the same type as those in the range of the class is to occur at that point in the string, except for one of the specific characters indicated after the $\wedge$ sign. For instance,

$$[\wedge D - Z][0 - 9][0 - 9]^*$$

stands for any string starting with a letter of the alphabet different from D to Z , followed by any positive number of digits from 0 to 9. Examples are $B3097$, $C0046$, and so forth. If r is a regular expression, the notation $r+$ denotes the concatenation of r with itself any positive finite number of times. In symbols,

$$r+ = rr^*.$$

For example,

$$[A - Z]^+$$

represents any nonempty string of capital letters. If r is a regular expression, then

$$r? = (\lambda \mid r).$$

That is, $r?$ denotes either zero occurrences or exactly one occurrence of r . Finally, if m and n are positive integers with $m \leq n$,

$r\{n\}$ denotes the concatenation of r with itself exactly n times,

and

$r\{m, n\}$ denotes the concatenation of r with itself anywhere from m through n times.

TEST YOURSELF

Answers to Test Yourself questions are located at the end of each section.

1. If x and y are strings, the concatenation of x and y is _____.
2. If L and L' are languages, the concatenation of L and L' is _____.
3. If L and L' are languages, the union of L and L' is _____.
4. If L is a language, the Kleene closure of L is _____.
5. The set of regular expressions over an alphabet Σ is defined recursively. The base for the definition is the statement that _____. The recursion for the definition specifies that if r and s are any regular expressions over Σ , then the following are also regular expressions in the set: _____, _____, and _____.
6. The function that associates a language to each regular expression over an alphabet Σ is defined recursively. The base for the definition is the statement that $L(\emptyset) =$ _____, $L(\lambda) =$ _____, and $L(a) =$ _____ for every a in Σ . The recursion for the definition specifies that if $L(r)$ and $L(r')$ are the languages defined by the regular expressions r and r' over Σ , then $L(rr') =$ _____, $L(r \mid r') =$ _____, and $L(r^*) =$ _____.
7. The notation $[A - C]$ is an example of a _____ and denotes the regular expression _____.
8. Use of a single dot in a regular expression stands for _____.
9. The symbol $\wedge$, placed at the beginning of a character class, indicates _____.

EXERCISE SET 12.1*

In 1 and 2, let $\Sigma = \{x, y\}$ be an alphabet.

1.
 - a. Let L_1 be the language consisting of all strings over Σ that are palindromes and have length ≤ 4 . List the elements of L_1 between braces.
 - b. Let L_2 be the language consisting of all strings over Σ that begin with an x and have length ≤ 3 . List the elements of L_2 .
2.
 - a. Let L_3 be the language consisting of all strings over Σ of length ≤ 3 in which all the x 's appear to the left of all the y 's. List the elements of L_3 between braces.
 - b. List between braces the elements of Σ^4 , the set of all strings of length 4 over Σ .
 - c. Let $A = \Sigma^1 \cup \Sigma^1$ and $B = \Sigma^3 \cup \Sigma^4$. Describe A , B , and $A \cup B$ in words.
3.
 - a. If the expression $ab + cd + \cdot$ in postfix notation is converted to infix notation, what is the result?
 - H b.** Let $\Sigma = \{1, 2, *, /\}$ and let L be the set of all strings over Σ obtained by writing first a number (1 or 2), then a second number (1 or 2), which can be the same as the first one, and finally an operation (denoted $*$ or $/$, where $*$ indicates multiplication and $/$ indicates division). Then L is a set of postfix, or reverse Polish, expressions. List all the elements of L between braces, and evaluate the resulting expressions.

In 4–6, describe L_1L_2 , $L_1 \cup L_2$, and $(L_1 \cup L_2)^*$ for the given languages L_1 and L_2 .

4. L_1 is the set of all strings of a 's and b 's that start with an a and contain only that one a ; L_2 is the set of all strings of a 's and b 's that contain an even number of a 's.
5. L_1 is the set of all strings of a 's, b 's, and c 's that contain no c 's and have the same number of a 's as b 's; L_2 is the set of all strings of a 's, b 's, and c 's that contain no a 's or b 's.
6. L_1 is the set of all strings of 0's and 1's that start with a 0; L_2 is the set of all strings of 0's and 1's that end with a 0.

In 7–9, add parentheses to emphasize the order of precedence in the given expressions.

7. $(a \mid b^*b)(a^* \mid ab)$ **8.** $0^*1 \mid 0(0^*1)^*$

9. $(x \mid yz^*)^*(yx \mid (yz)^*z)$

In 10–12, use the rules about order of precedence to eliminate the parentheses in the given regular expression.

10. $((a(b^*)) \mid (c(b^*))) ((ac) \mid (bc))$

11. $(1(1^*)) \mid ((1(0^*)) \mid ((1^*)1))$

12. $(xy)((x^*)y)^* \mid (((yx) \mid y)(y^*))$

In 13–15, use set notation to derive the language defined by the given regular expression. Assume $\Sigma = \{a, b, c\}$.

13. $\lambda \mid ab$

14. $\emptyset \mid \lambda$

15. $(a \mid b)c$

In 16–18, write five strings that belong to the language defined by the given regular expression.

- 16.** $0^*1(0^*1^*)^*$ **17.** $b^* \mid b^*ab^*$ **18.** $x^*(yxxxy \mid x)^*$

In 19–21, use words to describe the language defined by the given regular expression.

- 19.** $b^*ab^*ab^*a$ **20.** $1(0 \mid 1)^*00$ **21.** $(x \mid y)y(x \mid y)^*$

In 22–24, indicate whether the given strings belong to the language defined by the given regular expression. Briefly justify your answers.

- 22.** Expression: $(b \mid \lambda)a(a \mid b)^*a(b \mid \lambda)$, strings: *aaaba*, *baabb*
- 23.** Expression: $(x^*y \mid zy^*)^*$, strings: *zyyxz*, *zyyzy*
- 24.** Expression: $(01^*2)^*$, strings: *120*, *01202*

In 25–27, find a regular expression that defines the given language.

- 25.** The language consisting of all strings of 0's and 1's with an odd number of 1's. (Such a string is said to have *odd parity*.)
- 26.** The language consisting of all strings of a 's and b 's in which the third character from the end is a b .
- 27.** The language consisting of strings of x 's and y 's in which the elements in every pair of x 's are separated by at least one y .

Let r , s , and t be regular expressions over $\Sigma = \{a, b\}$. In 28–30, determine whether the two regular expressions define the same language. If they do, describe the language. If they do not, give an example of a string that is in one of the languages but not the other.

28. $(r \mid s)t$ and $rt \mid st$

29. $(rs)^*$ and r^*s^*

30. $(rs)^*$ and $((rs)^*)^*$

In 31–39, write a regular expression to define the given set of strings. Use the shorthand notations given in the section whenever convenient. In most cases, your expression will describe other strings in addition to the given ones, but try to make your answer fit the given strings as closely as possible within reasonable space limitations.

- 31.** All words that are written in lowercase letters and start with the letters *pre* but do not consist of *pre* all by itself.
- 32.** All words that are written in uppercase letters, and contain the letters *BIO* (as a unit) or *INFO* (as a unit).
- 33.** All words that are written in lowercase letters, end in *ly*, and contain at least five letters.

- 34.** All words that are written in lowercase letters and contain at least one of the vowels a, e, i, o, or u.
- 35.** All words that are written in lowercase letters and contain exactly one of the vowels a, e, i, o, or u.
- 36.** All words that are written in uppercase letters and do not start with one of the vowels A, E, I, O, or U but contain exactly two of these vowels next to each other.
- 37.** All United States social security numbers (which consist of three digits, a hyphen, two digits, another hyphen, and finally four more digits), where the final four digits start with a 3 and end with a 6.
- 38.** All telephone numbers that have three digits, then a hyphen, then three more digits, then a hyphen, and then four digits, where the first three digits are either 800 or 888 and the last four digits start and end with a 2.



Finite-State Automata

Example 12.2.1

A Simple Vending Machine

A simple vending machine dispenses bottles of juice that cost \$1 each. The machine accepts quarters and half-dollars only and does not give change. As soon as the amount deposited equals or exceeds \$1 the machine releases a bottle of juice. The next coin deposited starts the process over again. The operation of the machine is represented by the diagram of Figure 12.2.1.

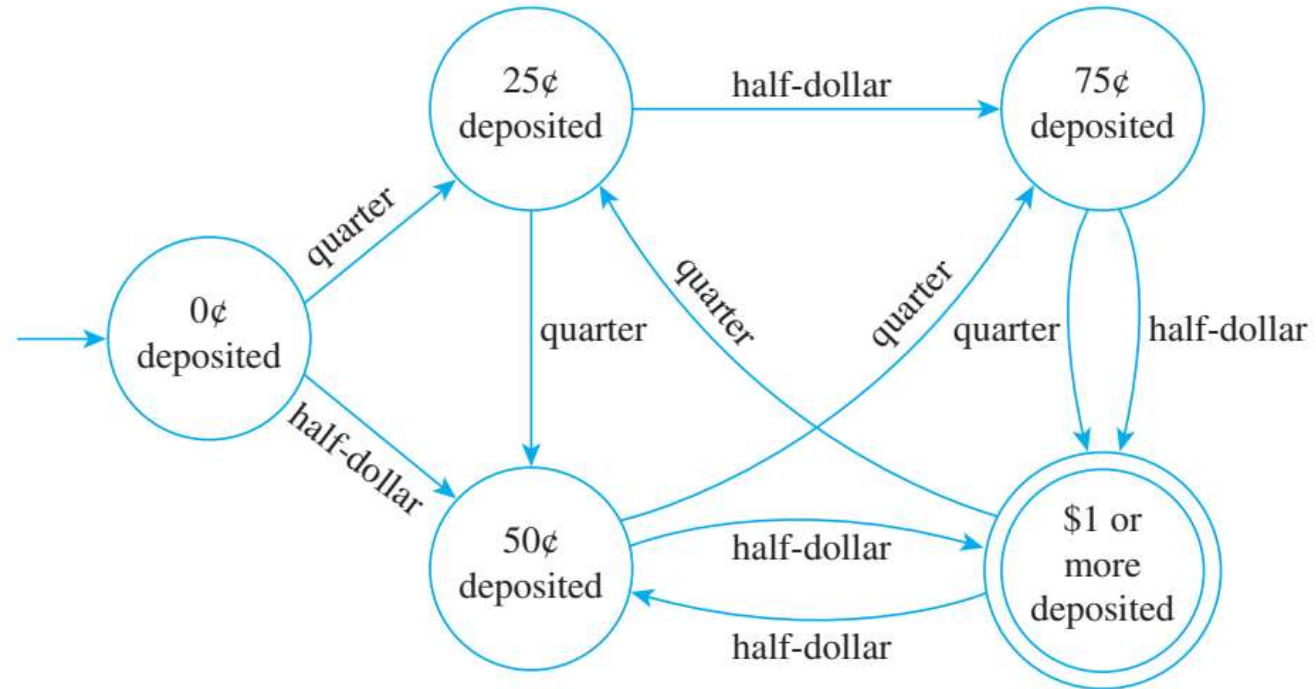


FIGURE 12.2.1 A Simple Vending Machine

Equivalently, the operation of the vending machine can be represented by a *next-state table* as shown in Table 12.2.1.

TABLE 12.2.1 Next-State Table

		Input	
		Quarter	Half-Dollar
→ State	0¢ deposited	25¢ deposited	50¢ deposited
	25¢ deposited	50¢ deposited	75¢ deposited
	50¢ deposited	75¢ deposited	\$1 or more deposited
	75¢ deposited	\$1 or more deposited	\$1 or more deposited
	⊙ \$1 or more deposited	25¢ deposited	50¢ deposited

Definition of a Finite-State Automaton

Definition

A **finite-state automaton** A consists of five objects:

1. A finite set I , called the **input alphabet**, of input symbols.
2. A finite set S of **states** the automaton can assume.
3. A designated state s_0 called the **initial state**.
4. A designated set of states called the set of **accepting states**.
5. A **next-state function** $N: S \times I \rightarrow S$ that associates a “next-state” to each ordered pair consisting of a “current state” and a “current input.” For each state s in S and input symbol m in I , $N(s, m)$ is the state to which A goes if m is input to A when A is in state s .

The operation of a finite-state automaton is commonly described by a diagram called a **(state-) transition diagram**, similar to that of Figure 12.2.1. It is called a *transition diagram* because it shows the transitions the machine makes from one state to another in response to various inputs. In a transition diagram, states are represented by circles and accepting states by double circles. There is one arrow that points to the initial state and there are other arrows that are labeled with input symbols and point from each state to other states to indicate the action of the next-state function. Specifically, an arrow from state s to state t labeled m means that $N(s, m) = t$.

The **next-state table** for an automaton shows the values of the next-state function N for all possible states s and input symbols i . In the **annotated next-state table**, the initial state is indicated by an arrow and the accepting states are marked by double circles.

Example 12.2.2 A Finite-State Automaton Given by a Transition Diagram

Consider the finite-state automaton A defined by the transition diagram shown in Figure 12.2.2.

- What are the states of A ?
- What are the input symbols of A ?
- What is the initial state of A ?
- What are the accepting states of A ?
- Determine $N(s_1, 1)$.
- Construct the annotated next-state table for A .

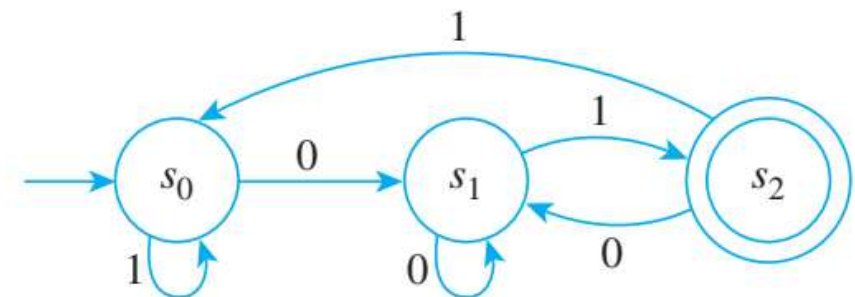


FIGURE 12.2.2

Solution

- The states of A are s_0 , s_1 , and s_2 [since these are the labels of the circles].
- The input symbols of A are 0 and 1 [since these are the labels of the arrows].
- The initial state of A is s_0 [since the unlabeled arrow points to s_0].
- The only accepting state of A is s_2 [since this is the only state marked by a double circle].
- $N(s_1, 1) = s_2$ [since there is an arrow from s_1 to s_2 labeled 1]

f.

		Input	
		0	1
State	$\rightarrow$	s_1	s_0
	s_1	s_1	s_2
	$\odot$ s_2	s_1	s_0

Example 12.2.3

A Finite-State Automaton Given by an Annotated Next-State Table

Consider the finite-state automaton A defined by the following annotated next-state table:

- What are the states of A ?
- What are the input symbols of A ?
- What is the initial state of A ?
- What are the accepting states of A ?
- Find $N(U, c)$.
- Draw the transition diagram for A .

		Input			
		a	b	c	
State	$\rightarrow$	U	Z	Y	Y
	$\odot$	V	V	V	V
		Y	Z	V	Y
	$\odot$	Z	Z	Z	Z

Solution

- a. The states of A are U , V , Y , and Z .
- b. The input symbols of A are a , b , and c .
- c. The initial state of A is U [since the arrow points to U].
- d. The accepting states of A are V and Z [since these are marked with double circles].
- e. $N(U, c) = Y$ [since the entry in the row labeled U and the column labeled c of the next-state table is Y].
- f. The transition diagram for A is shown in Figure 12.2.3. It can be drawn more compactly by labeling arrows with multiple-input symbols where appropriate. This is illustrated in Figure 12.2.4.

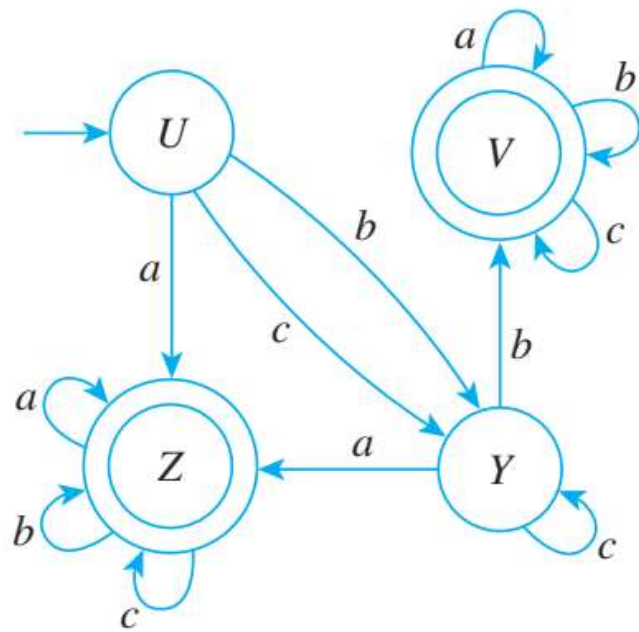


FIGURE 12.2.3

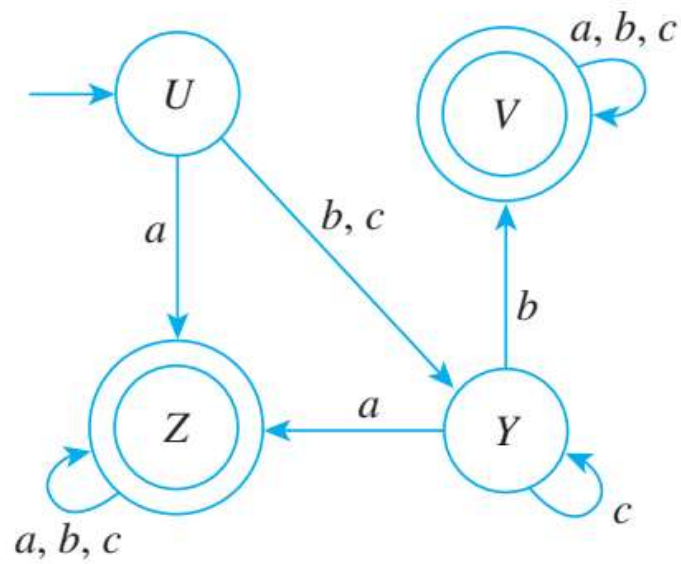


FIGURE 12.2.4



TEST YOURSELF

1. The five objects that make up a finite-state automaton are _____, _____, _____, _____, and _____.
2. The next-state table for an automaton shows the values of _____.
3. In the annotated next-state table, the initial state is indicated with an _____ and the accepting states are marked by _____.
4. A string w consisting of input symbols is accepted by a finite-state automaton A if, and only if, _____.
5. The language accepted by a finite-state automaton A is _____.
6. If N is the next-state function for a finite-state automaton A , the eventual-state function N^* is defined as follows: For each state s of A and for each string w that consists of input symbols of A , $N^*(s, w) = \underline{\hspace{2cm}}$.
7. One part of Kleene's theorem says that given any language that is accepted by a finite-state automaton, there is _____.
8. The second part of Kleene's theorem says that given any language defined by a regular expression, there is _____.
9. A regular language is _____.
10. Given the language consisting of all strings of the form $a^k b^k$, where k is a positive integer, the pigeonhole principle can be used to show that the language is _____.

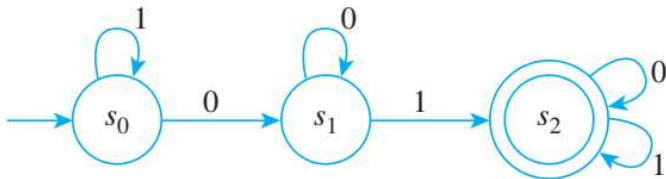
EXERCISE SET 12.2

1. Find the state of the vending machine in Example 12.2.1 after each of the following sequences of coins have been input.
 - a. Quarter, half-dollar, quarter
 - b. Quarter, half-dollar, half-dollar
 - c. Half-dollar, quarter, quarter, quarter, half-dollar

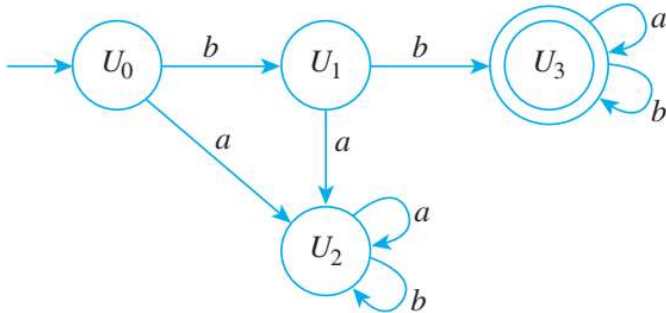
In 2–7, a finite-state automaton is given by a transition diagram. For each automaton:

- Find its states.
- Find its input symbols.
- Find its initial state.
- Find its accepting states.
- Write its annotated next-state table.

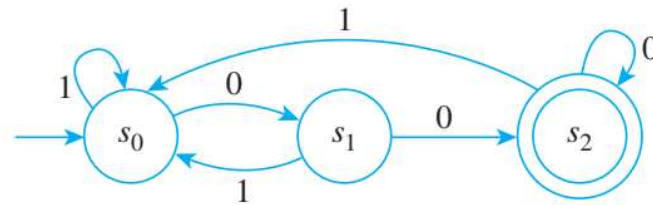
2.



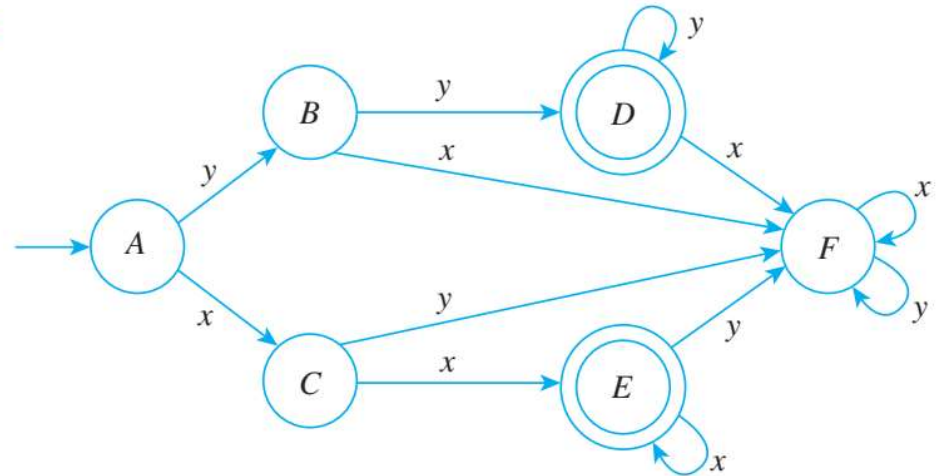
3.



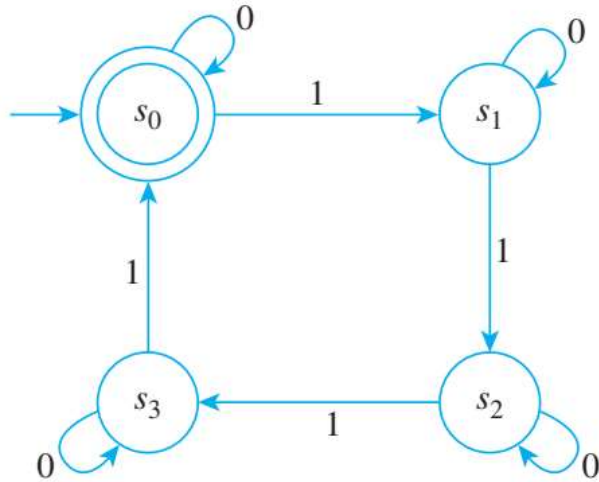
4.



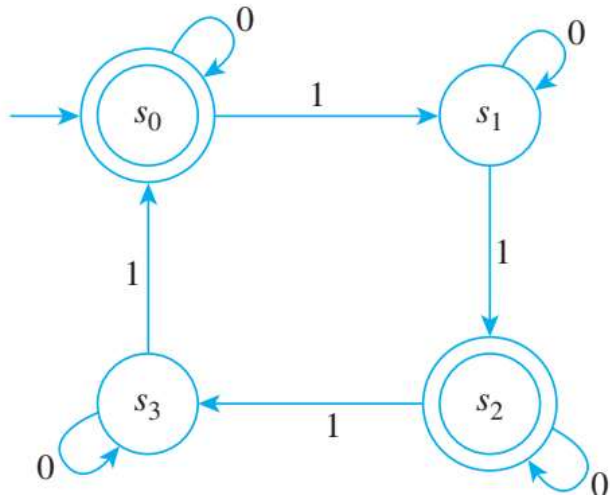
5.



6.



7.



In 8 and 9, a finite-state automaton is given by an annotated next-state table. For each automaton:

- Find its states.
- Find its input symbols.
- Find its initial state.
- Find its accepting states.
- Draw its transition diagram.

8. Next-State Table

		Input	
		0	1
State	→	s_0	s_1
	⊙	s_1	s_2
	⊙	s_2	s_1

9. Next-State Table

		Input	
		0	1
State	→	s_0	s_0
	⊙	s_1	s_1
	⊙	s_2	s_2
	⊙	s_3	s_3



The Language Accepted by an Automaton

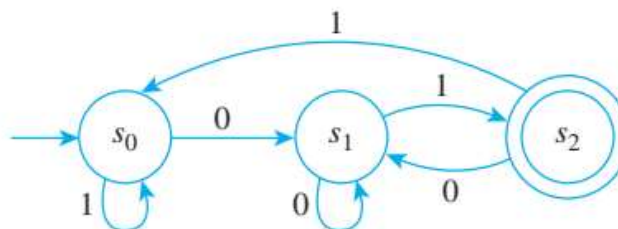
Definition

Let A be a finite-state automaton with set of input symbols I . Let I^* be the set of all strings over I , and let w be a string in I^* . Then w is **accepted by A** if, and only if, A goes to an accepting state when the symbols of w are input to A in sequence from left to right, starting when A is in its initial state. **The language accepted by A , denoted $L(A)$, is the set of all strings that are accepted by A .**

Example 12.2.4

Finding the Language Accepted by an Automaton

Consider the finite-state automaton A defined in Example 12.2.2 and shown again below.



- To what states does A go if the symbols of the following strings are input to A in sequence, starting from the initial state?
 (i) 01 (ii) 0011 (iii) 0101100 (iv) 10101
- Which of the strings in part (a) send A to an accepting state?
- What is the language accepted by A ?
- Is there a regular expression that defines the same language?

Solution

- a. (i) s_2 (ii) s_0 (iii) s_1 (iv) s_2
- b. The strings 01 and 10101 send A to an accepting state.
- c. Observe that if w is any string that ends in 01, then w is accepted by A . For if w is any string of length $n \geq 2$, then after the first $n - 2$ symbols of w have been input, A is in one of its three states: s_0 , s_1 , or s_2 . But from any of these three states, input of the symbols 01 in sequence sends A first to s_1 and then to the accepting state s_2 . Hence any string that ends in 01 is accepted by A .

Also note that the only strings accepted by A are those that end in 01. (That is, no other strings besides those ending in 01 are accepted by A .) The reason is that the only accepting state of A is s_2 , and the only arrow pointing to s_2 comes from s_1 and is labeled 1. Thus in order for an input string w of length n to send A to an accepting state, the last symbol of w must be a 1 and the first $n - 1$ symbols of w must send A to state s_1 . Now three arrows point to s_1 , one from each of the three states of A , and all are labeled 0. Thus the last of the first $n - 1$ symbols of w must be 0, or, in other words, the next-to-the-last symbol of w must be 0. Hence the last two symbols of w must be 01, and thus

$$L(A) = \text{the set of all strings of 0's and 1's that end in 01.}$$

- d. Yes. One regular expression that defines $L(A)$ is $(0 \mid 1)^*01$. ■

The Eventual-State Function

- Now suppose a finite-state automaton is in one of its states (not necessarily the initial state) and a string of input symbols is fed into it in sequence. To what state will the automaton eventually go?
- The function that gives the answer to this question for every possible combination of input strings and states of the automaton is called the *eventual-state function*

Definition

Let A be a finite-state automaton with set of input symbols I , set of states S , and next-state function $N: S \times I \rightarrow S$. Let I^* be the set of all strings over I , and define the **eventual-state function** $N^*: S \times I^* \rightarrow S$ as follows:

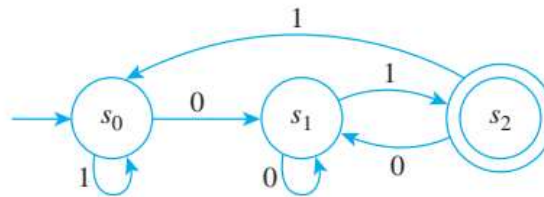
For any state s and for any input string w ,

$$N^*(s, w) = \left[\begin{array}{l} \text{the state to which } A \text{ goes if the} \\ \text{symbols of } w \text{ are input to } A \text{ in sequence,} \\ \text{starting when } A \text{ is in state } s \end{array} \right].$$

Example 12.2.5

Computing Values of the Eventual-State Function

Consider again the finite-state automaton of Example 12.2.2 shown below for convenience. Find $N^*(s_1, 10110)$.



Solution By definition of the eventual-state function,

$$N^*(s_1, 10110) = \left[\begin{array}{l} \text{the state to which } A \text{ goes if the} \\ \text{symbols of } 10110 \text{ are input to } A \text{ in} \\ \text{sequence, starting when } A \text{ is in state } s_1 \end{array} \right].$$

By referring to the transition diagram for A , you can see that starting from s_1 , when a 1 is input, A goes to s_2 ; then when a 0 is input, A goes back to s_1 ; after that, when a 1 is input, A goes to s_2 ; from there, when a 1 is input, A goes to s_0 ; and finally, when a 0 is input, A goes back to s_1 . This sequence of state transitions can be written as follows:

$$s_1 \xrightarrow{1} s_2 \xrightarrow{0} s_1 \xrightarrow{1} s_2 \xrightarrow{1} s_0 \xrightarrow{0} s_1.$$

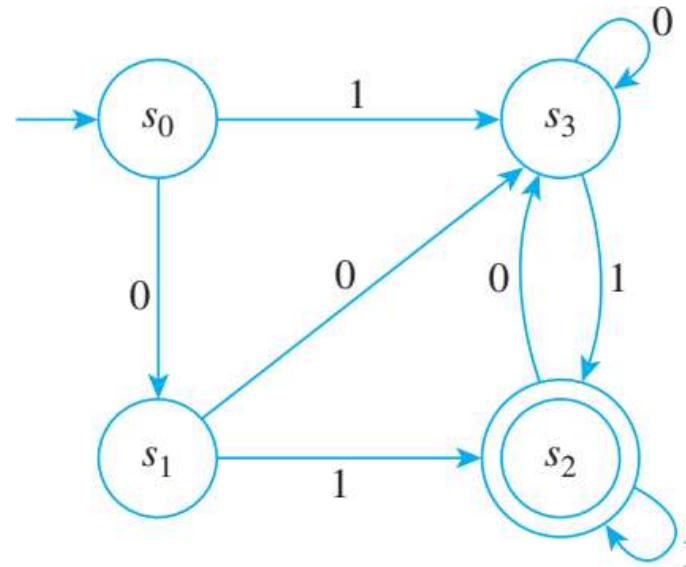
Thus, after all the symbols of 10110 have been input in sequence, the eventual state of A is s_1 , so

$$N^*(s_1, 10110) = s_1.$$

w is accepted by $A \iff N^*(s_0, w)$ is an accepting state of A

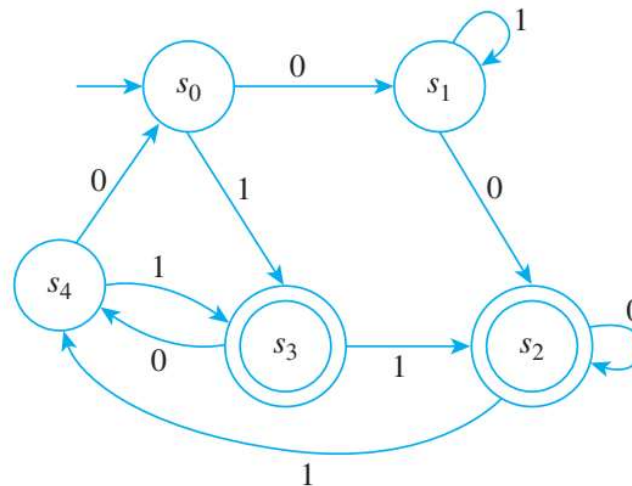
$L(A) = \{w \in I^* \mid N^*(s_0, w) \text{ is an accepting state of } A\}$

- 10.** A finite-state automaton A , given by the transition diagram below, has next-state function N and eventual-state function N^* .



- Find $N(s_1, 1)$ and $N(s_0, 1)$.
- Find $N(s_2, 0)$ and $N(s_1, 0)$.
- Find $N^*(s_0, 10011)$ and $N^*(s_1, 01001)$.
- Find $N^*(s_2, 11010)$ and $N^*(s_0, 01000)$.

- 11.** A finite-state automaton A , given by the transition diagram on the next page, has next-state function N and eventual-state function N^* .



- Find $N(s_3, 0)$ and $N(s_2, 1)$.
- Find $N(s_0, 0)$ and $N(s_4, 1)$.
- Find $N^*(s_0, 010011)$ and $N^*(s_3, 01101)$.
- Find $N^*(s_0, 1111)$ and $N^*(s_2, 00111)$.



Designing a Finite-State Automaton

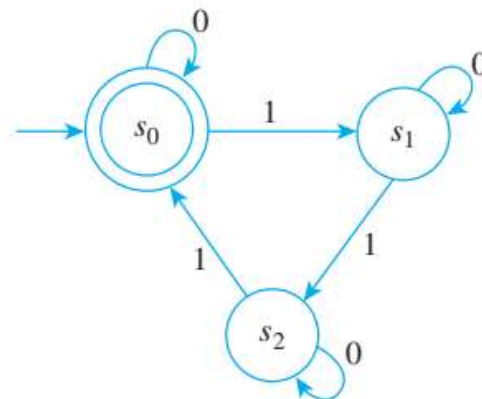
Example 12.2.6

**A Finite-State Automaton That Accepts the Set of Strings of 0's and 1's
for Which the Number of 1's Is Divisible by 3**

- a. Design a finite-state automaton A that accepts the set of all strings of 0's and 1's such that the number of 1's in the string is divisible by 3.
- b. Is there a regular expression that defines this set?

□ $s(k)$ = the state of a string where the remainder of the number of '1' in s dividing 3 equals to k .

It follows that the transition diagram for A has the appearance shown below.



This automaton accepts the set of strings of 0's and 1's for which the number of 1's is divisible by 3.

b. A regular expression that defines the given set is $0^*|(0^*10^*10^*10^*)^*$.



Example 12.2.7

A Finite-State Automaton That Accepts the Set of All Strings of 0's and 1's Containing Exactly One 1

- a. Design a finite-state automaton A to accept the set of all strings of 0's and 1's that contain exactly one 1.
- b. Is there a regular expression that defines this set?

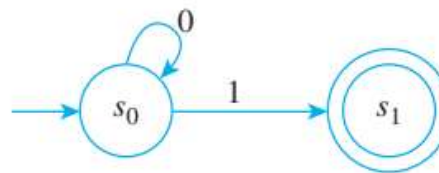
Solution

- a. The automaton A must have at least two distinct states:

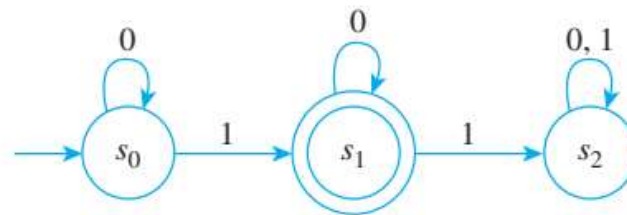
s_0 : initial state;

s_1 : state to which A goes when the input string contains exactly one 1.

If A is in state s_0 and a 0 is input, A may as well stay in state s_0 (since it still needs to wait for a 1 to move to state s_1), but as soon as a 1 is input, A moves to state s_1 . Thus a partial drawing of the transition diagram is as shown below.



Now consider what happens when A is in state s_1 . If a 0 is input, the input string still has a single 1, so A stays in state s_1 . But if a 1 is input, then the input string contains more than one 1, so A must leave s_1 (since no string with more than one 1 is to be accepted by A). It cannot go back to state s_0 because there is a way to get from s_0 to s_1 , and after input of the second 1, A can never return to state s_1 . Hence A must go to a third state, s_2 , from which there is no return to s_1 . Thus from s_2 every input may as well leave A in state s_2 . It follows that the completed transition diagram for A has the appearance shown below.



This automaton accepts the set of strings 0's and 1's, with exactly one 1.

- b. A regular expression that defines the given set is 0^*10^* .



- 12.** Consider again the finite-state automaton of exercise 2.
- a.** To what state does the automaton go when the symbols of the following strings are input to it in sequence, starting from the initial state?
 (i) 1110001 (ii) 0001000 (iii) 11110000
 - b.** Which of the strings in part (a) send the automaton to an accepting state?
 - c.** What is the language accepted by the automaton?
 - d.** Find a regular expression that defines the language.

- 13.** Consider again the finite-state automaton of exercise 3.
- a.** To what state does the automaton go when the symbols of the following strings are input to it in sequence, starting from the initial state?
 (i) *bb* (ii) *aabbbaba* (iii) *babbbbbabaa*
 (iv) *baaaaabaa*
 - b.** Which of the strings in part (a) send the automaton to an accepting state?
 - c.** What is the language accepted by the automaton?
 - d.** Find a regular expression that defines the language.

In each of 14–19, (a) find the language accepted by the automaton in the referenced exercise, and (b) find a regular expression that defines the same language.

- | | | |
|-----------------------|-----------------------|-----------------------|
| 14. Exercise 4 | 15. Exercise 5 | 16. Exercise 6 |
| 17. Exercise 7 | 18. Exercise 8 | 19. Exercise 9 |

In each of 20–28, (a) design an automaton with the given input alphabet that accepts the given set of strings, and

(b) find a regular expression that defines the language accepted by the automaton.

- 20. Input alphabet = $\{0, 1\}$; Accepts the set of all strings for which the final three input symbols are 1.
- 21. Input alphabet = $\{0, 1\}$; Accepts the set of all strings that start with 01.
- H 22. Input alphabet = $\{a, b\}$; Accepts the set of all strings of length at least 2 for which the final two input symbols are the same.
- 23. Input alphabet = $\{0, 1\}$; Accepts the set of all strings that start with 01 or 10.
- 24. Input alphabet = $\{0, 1\}$; Accepts the set of all strings that start with 101.
- 25. Input alphabet = $\{0, 1\}$; Accepts the set of all strings that end in 10.
- 26. Input alphabet = $\{a, b\}$; Accepts the set of all strings that contain exactly two b 's.
- 27. Input alphabet = $\{0, 1\}$; Accepts the set of all strings that start with 0 and contain exactly one 1.
- 28. Input alphabet = $\{0, 1\}$; Accepts the set of all strings that contain the pattern 010.



13.1 Languages and Grammars

13.1.1 Introduction

- Words in the English language can be combined in various ways.
- **natural language**, that is, a spoken language, such as English, French, German, or Spanish, is extremely complicated
- a **formal language**, which, unlike a natural language, is specified by a well-defined set of rules of syntax

13.1.2 Phrase-Structure Grammars

Definition 1

A *vocabulary* (or *alphabet*) V is a finite, nonempty set of elements called *symbols*. A *word* (or *sentence*) over V is a string of finite length of elements of V . The *empty string* or *null string*, denoted by λ (and sometimes by ϵ), is the string containing no symbols. The set of all words over V is denoted by V^* . A *language over V* is a subset of V^* .

Definition 2

A *phrase-structure grammar* $G = (V, T, S, P)$ consists of a vocabulary V , a subset T of V consisting of terminal symbols, a start symbol S from V , and a finite set of productions P . The set $V - T$ is denoted by N . Elements of N are called *nonterminal symbols*. Every production in P must contain at least one nonterminal on its left side.

EXAMPLE 1

Let $G = (V, T, S, P)$, where $V = \{a, b, A, B, S\}$, $T = \{a, b\}$, S is the start symbol, and $P = \{S \rightarrow ABa, A \rightarrow BB, B \rightarrow ab, AB \rightarrow b\}$. G is an example of a phrase-structure grammar. 

Definition 3

Let $G = (V, T, S, P)$ be a phrase-structure grammar. Let $w_0 = lz_0r$ (that is, the concatenation of l , z_0 , and r) and $w_1 = lz_1r$ be strings over V . If $z_0 \rightarrow z_1$ is a production of G , we say that w_1 is *directly derivable* from w_0 and we write $w_0 \Rightarrow w_1$. If $w_0, w_1, \dots, w_n$ are strings over V such that $w_0 \Rightarrow w_1, w_1 \Rightarrow w_2, \dots, w_{n-1} \Rightarrow w_n$, then we say that w_n is *derivable from* w_0 , and we write $w_0 \xRightarrow{*} w_n$. The sequence of steps used to obtain w_n from w_0 is called a *derivation*.

EXAMPLE 2

The string $Aaba$ is directly derivable from ABa in the grammar in Example 1 because $B \rightarrow ab$ is a production in the grammar. The string $abababa$ is derivable from ABa because $ABa \Rightarrow Aaba \Rightarrow BBaba \Rightarrow Bababa \Rightarrow abababa$, using the productions $B \rightarrow ab$, $A \rightarrow BB$, $B \rightarrow ab$, and $B \rightarrow ab$ in succession. 

Definition 4

Let $G = (V, T, S, P)$ be a phrase-structure grammar. The *language generated by G* (or the *language of G*), denoted by $L(G)$, is the set of all strings of terminals that are derivable from the starting state S . In other words,

$$L(G) = \{w \in T^* \mid S \xRightarrow{*} w\}.$$

EXAMPLE 3

Extra
Examples

Let G be the grammar with vocabulary $V = \{S, A, a, b\}$, set of terminals $T = \{a, b\}$, starting symbol S , and productions $P = \{S \rightarrow aA, S \rightarrow b, A \rightarrow aa\}$. What is $L(G)$, the language of this grammar?

Solution: From the start state S we can derive aA using the production $S \rightarrow aA$. We can also use the production $S \rightarrow b$ to derive b . From aA the production $A \rightarrow aa$ can be used to derive aaa . No additional words can be derived. Hence, $L(G) = \{b, aaa\}$. ◀

EXAMPLE 4 Let G be the grammar with vocabulary $V = \{S, 0, 1\}$, set of terminals $T = \{0, 1\}$, starting symbol S , and productions $P = \{S \rightarrow 11S, S \rightarrow 0\}$. What is $L(G)$, the language of this grammar?

Solution: From S we can derive 0 using $S \rightarrow 0$, or $11S$ using $S \rightarrow 11S$. From $11S$ we can derive either 110 or $1111S$. From $1111S$ we can derive 11110 and $111111S$. At any stage of a derivation we can either add two 1s at the end of the string or terminate the derivation by adding a 0 at the end of the string. We surmise that $L(G) = \{0, 110, 11110, 1111110, \dots\}$, the set of all strings that begin with an even number of 1s and end with a 0. This can be proved using an inductive argument that shows that after n productions have been used, the only strings of terminals generated are those consisting of $n - 1$ concatenations of 11 followed by 0. (This is left as an exercise for the reader.) 

EXAMPLE 5 Give a phrase-structure grammar that generates the set $\{0^n 1^n \mid n = 0, 1, 2, \dots\}$.

Solution: Two productions can be used to generate all strings consisting of a string of 0s followed by a string of the same number of 1s, including the null string. The first builds up successively longer strings in the language by adding a 0 at the start of the string and a 1 at the end. The second production replaces S with the empty string. The solution is the grammar $G = (V, T, S, P)$, where $V = \{0, 1, S\}$, $T = \{0, 1\}$, S is the starting symbol, and the productions are

$$\begin{aligned} S &\rightarrow 0S1 \\ S &\rightarrow \lambda. \end{aligned}$$

The verification that this grammar generates the correct set is left as an exercise for the reader. 

EXAMPLE 6 Find a phrase-structure grammar to generate the set $\{0^m 1^n \mid m \text{ and } n \text{ are nonnegative integers}\}$.

Solution: We will give two grammars G_1 and G_2 that generate this set. This will illustrate that two different grammars can generate the same language.

The grammar G_1 has alphabet $V = \{S, 0, 1\}$; terminals $T = \{0, 1\}$; and productions $S \rightarrow 0S$, $S \rightarrow S1$, and $S \rightarrow \lambda$. Then, G_1 generates the correct set, because using the first production m times puts m 0s at the beginning of the string, and using the second production n times puts n 1s at the end of the string. Furthermore, every string produced by this grammar is of the form $0^m 1^n$, because the only way to add a symbol to the beginning of a string is to add a 0 by applying the production $S \rightarrow 0S$, and the only way to add a symbol to the end of the string is to add a 1 by applying the production $S \rightarrow S1$.

The grammar G_2 has alphabet $V = \{S, A, 0, 1\}$; terminals $T = \{0, 1\}$; and productions $S \rightarrow 0S$, $S \rightarrow 1A$, $S \rightarrow 1$, $A \rightarrow 1A$, $A \rightarrow 1$, and $S \rightarrow \lambda$. The details that this grammar generates the correct set are left as an exercise for the reader. 

EXAMPLE 7 One grammar that generates the set $\{0^n 1^n 2^n \mid n = 0, 1, 2, 3, \dots\}$ is $G = (V, T, S, P)$ with $V = \{0, 1, 2, S, A, B, C\}$; $T = \{0, 1, 2\}$; starting state S ; and productions $S \rightarrow C$, $C \rightarrow 0CAB$, $S \rightarrow \lambda$, $BA \rightarrow AB$, $0A \rightarrow 01$, $1A \rightarrow 11$, $1B \rightarrow 12$, and $2B \rightarrow 22$. We leave it as an exercise for the reader (Exercise 12) to show that this statement is correct. The grammar given is the simplest type of grammar that generates this set, in a sense that will be made clear later in this section. 

13.1.3 Types of Phrase-Structure Grammars

- ❑ A **type 0** grammar has no restrictions on its productions.
- ❑ A **type 1** grammar can have productions of the form $w_1 \rightarrow w_2$, where $w_1 = lAr$ and $w_2 = lwr$, where A is a nonterminal symbol, l and r are strings of zero or more terminal or nonterminal symbols, and w is a nonempty string of terminal or nonterminal symbols. It can also have the production $S \rightarrow \lambda$ as long as S does not appear on the right-hand side of any other production.
- ❑ A **type 2** grammar can have productions only of the form $w_1 \rightarrow w_2$, where w_1 is a single symbol that is not a terminal symbol
- ❑ **type 3** grammar can have productions only of the form $w_1 \rightarrow w_2$ with $w_1 = A$ and either $w_2 = aB$ or $w_2 = a$, where A and B are nonterminal symbols and a is a terminal symbol, or with $w_1 = S$ and $w_2 = \lambda$

13.1.3 Types of Phrase-Structure Grammars

- Type 2 grammars are called **context-free grammars** because a nonterminal symbol that is the left side of a production can be replaced in a string whenever it occurs, no matter what else is in the string.
 - A language generated by a type 2 grammar is called a **context-free language**.
- When there is a production of the form $lw_1r \rightarrow lw_2r$ (but not of the form $w_1 \rightarrow w_2$), the grammar is called type 1 or **context-sensitive** because w_1 can be replaced by w_2 only when it is surrounded by the strings l and r .
 - A language generated by a type 1 grammar is called a **context-sensitive language**.
- Type 3 grammars are also called **regular grammars**. A language generated by a regular grammar is called **regular**.

- EXAMPLE 8** From Example 6 we know that $\{0^m 1^n \mid m, n = 0, 1, 2, \dots\}$ is a regular language, because it can be generated by a regular grammar, namely, the grammar G_2 in Example 6. ◀
- EXAMPLE 9** It follows from Example 5 that $\{0^n 1^n \mid n = 0, 1, 2, \dots\}$ is a context-free language, because the productions in this grammar are $S \rightarrow 0S1$ and $S \rightarrow \lambda$. However, it is not a regular language. This will be shown in Section 13.4. ◀
- EXAMPLE 10** The set $\{0^n 1^n 2^n \mid n = 0, 1, 2, \dots\}$ is a context-sensitive language, because it can be generated by a type 1 grammar, as Example 7 shows, but not by any type 2 language. (This is shown in Exercise 28 in the supplementary exercises at the end of the chapter.) ◀

4. Let $G = (V, T, S, P)$ be the phrase-structure grammar with $V = \{0, 1, A, S\}$, $T = \{0, 1\}$, and set of productions P consisting of $S \rightarrow 1S$, $S \rightarrow 00A$, $A \rightarrow 0A$, and $A \rightarrow 0$.
 - a) Show that 111000 belongs to the language generated by G .
 - b) Show that 11001 does not belong to the language generated by G .
 - c) What is the language generated by G ?
5. Let $G = (V, T, S, P)$ be the phrase-structure grammar with $V = \{0, 1, A, B, S\}$, $T = \{0, 1\}$, and set of productions P consisting of $S \rightarrow 0A$, $S \rightarrow 1A$, $A \rightarrow 0B$, $B \rightarrow 1A$, $B \rightarrow 1$.
 - a) Show that 10101 belongs to the language generated by G .
 - b) Show that 10110 does not belong to the language generated by G .
 - c) What is the language generated by G ?
- *6. Let $V = \{S, A, B, a, b\}$ and $T = \{a, b\}$. Find the language generated by the grammar (V, T, S, P) when the set P of productions consists of
 - a) $S \rightarrow AB, A \rightarrow ab, B \rightarrow bb$.
 - b) $S \rightarrow AB, S \rightarrow aA, A \rightarrow a, B \rightarrow ba$.
 - c) $S \rightarrow AB, S \rightarrow AA, A \rightarrow aB, A \rightarrow ab, B \rightarrow b$.
 - d) $S \rightarrow AA, S \rightarrow B, A \rightarrow aaA, A \rightarrow aa, B \rightarrow bB, B \rightarrow b$.
 - e) $S \rightarrow AB, A \rightarrow aAb, B \rightarrow bBa, A \rightarrow \lambda, B \rightarrow \lambda$.

7. Construct a derivation of $0^3 1^3$ using the grammar given in Example 5.
8. Show that the grammar given in Example 5 generates the set $\{0^n 1^n \mid n = 0, 1, 2, \dots\}$.
9. a) Construct a derivation of $0^2 1^4$ using the grammar G_1 in Example 6.
b) Construct a derivation of $0^2 1^4$ using the grammar G_2 in Example 6.
10. a) Show that the grammar G_1 given in Example 6 generates the set $\{0^m 1^n \mid m, n = 0, 1, 2, \dots\}$.
b) Show that the grammar G_2 in Example 6 generates the same set.
11. Construct a derivation of $0^2 1^2 2^2$ in the grammar given in Example 7.
- * 12. Show that the grammar given in Example 7 generates the set $\{0^n 1^n 2^n \mid n = 0, 1, 2, \dots\}$.
13. Find a phrase-structure grammar for each of these languages.
 - a) the set consisting of the bit strings 0, 1, and 11
 - b) the set of bit strings containing only 1s
 - c) the set of bit strings that start with 0 and end with 1
 - d) the set of bit strings that consist of a 0 followed by an even number of 1s
14. Find a phrase-structure grammar for each of these languages.
 - a) the set consisting of the bit strings 10, 01, and 101
 - b) the set of bit strings that start with 00 and end with one or more 1s
 - c) the set of bit strings consisting of an even number of 1s followed by a final 0
 - d) the set of bit strings that have neither two consecutive 0s nor two consecutive 1s
- * 15. Find a phrase-structure grammar for each of these languages.
 - a) the set of all bit strings containing an even number of 0s and no 1s
 - b) the set of all bit strings made up of a 1 followed by an odd number of 0s
 - c) the set of all bit strings containing an even number of 0s and an even number of 1s
 - d) the set of all strings containing 10 or more 0s and no 1s
 - e) the set of all strings containing more 0s than 1s
 - f) the set of all strings containing an equal number of 0s and 1s
 - g) the set of all strings containing an unequal number of 0s and 1s

16. Construct phrase-structure grammars to generate each of these sets.

- a) $\{1^n \mid n \geq 0\}$ b) $\{10^n \mid n \geq 0\}$
c) $\{(11)^n \mid n \geq 0\}$

17. Construct phrase-structure grammars to generate each of these sets.

- a) $\{0^n \mid n \geq 0\}$ b) $\{1^n 0 \mid n \geq 0\}$
c) $\{(000)^n \mid n \geq 0\}$

18. Construct phrase-structure grammars to generate each of these sets.

- a) $\{01^{2n} \mid n \geq 0\}$
b) $\{0^n 1^{2n} \mid n \geq 0\}$
c) $\{0^n 1^m 0^n \mid m \geq 0 \text{ and } n \geq 0\}$

19. Let $V = \{S, A, B, a, b\}$ and $T = \{a, b\}$. Determine whether $G = (V, T, S, P)$ is a type 0 grammar but not a type 1 grammar, a type 1 grammar but not a type 2 grammar, or a type 2 grammar but not a type 3 grammar if P , the set of productions, is

- a) $S \rightarrow aAB, A \rightarrow Bb, B \rightarrow \lambda$.
b) $S \rightarrow aA, A \rightarrow a, A \rightarrow b$.
c) $S \rightarrow ABa, AB \rightarrow a$.
d) $S \rightarrow ABA, A \rightarrow aB, B \rightarrow ab$.
e) $S \rightarrow bA, A \rightarrow B, B \rightarrow a$.
f) $S \rightarrow aA, aA \rightarrow B, B \rightarrow aA, A \rightarrow b$.
g) $S \rightarrow bA, A \rightarrow b, S \rightarrow \lambda$.
h) $S \rightarrow AB, B \rightarrow aAb, aAb \rightarrow b$.
i) $S \rightarrow aA, A \rightarrow bB, B \rightarrow b, B \rightarrow \lambda$.
j) $S \rightarrow A, A \rightarrow B, B \rightarrow \lambda$.

20. A **palindrome** is a string that reads the same backward as it does forward, that is, a string w , where $w = w^R$, where w^R is the reversal of the string w . Find a context-free grammar that generates the set of all palindromes over the alphabet $\{0, 1\}$.

